



Pontifícia Universidade Católica do Rio Grande do Sul
Fundamentos de Sistemas Computacionais
98800-04

Exercícios - Operações Bitwise e Masking

Prof. Iaçanã Ianiski Weber

Lembrete: Operadores Bitwise

Em linguagens como C e Java, os operadores bitwise atuam **bit a bit** sobre os operandos:

Operador	Nome	Operador	Nome
&	AND bitwise		OR bitwise
^	XOR bitwise	~	NOT (complemento)
«	Shift left	»	Shift right

Masking é a técnica de usar uma constante (máscara) combinada com um operador bitwise para ler, ligar, desligar ou alternar bits específicos de um valor, sem afetar os demais.

Questão 1 — Avaliação de expressões bitwise

Considere variáveis de **8 bits sem sinal**. Avalie cada expressão e dê o resultado em binário e hexadecimal.

- a) $0b11001010 \ \& \ 0b11110000$
- b) $0b10110011 \ |\ 0b00001111$
- c) $0b11010110 \ \wedge \ 0b11111111$
- d) $\sim 0b00110011$ (considere 8 bits)
- e) $0b00001101 \ \ll \ 3$
- f) $0b11010000 \ \gg \ 4$

Resposta — Questão 1

- a) $0b11000000 = 0xC0$
- b) $0b10111111 = 0xBF$
- c) $0b00101001 = 0x29$
- d) $0b11001100 = 0xCC$
- e) $0b01101000 = 0x68$
- f) $0b00001101 = 0x0D$

Questão 2 — Identificando a operação

Para cada par (valor original $\rightarrow$ resultado), determine qual operador bitwise e qual máscara foram usados. Todos os valores são de 8 bits.

- a) $0b10110110 \rightarrow 0b00110000$
- b) $0b10100000 \rightarrow 0b10100111$
- c) $0b11111011 \rightarrow 0b11110011$
- d) $0b10101010 \rightarrow 0b01010101$

Resposta — Questão 2

- a) AND com 0b00110000 (0x30): $0b10110110 \& 0x30 = 0b00110000$
- b) OR com 0b00000111 (0x07): $0b10100000 \mid 0x07 = 0b10100111$
- c) AND com 0b11110111 (0xF7), ou equivalentemente $\& \sim(1 \ll 3)$: limpa o bit 3.
- d) XOR com 0b11111111 (0xFF), ou equivalentemente NOT ($\sim$): inverte todos os bits.

Questão 3 — Máscaras para tarefas específicas

Para cada tarefa, escreva a expressão usando operadores bitwise. Considere uma variável `unsigned char x` (8 bits).

- a) Extrair (isolar) os 4 bits mais significativos de `x`.
- b) Ligar (set) o bit 5 de `x`, sem alterar os demais.
- c) Desligar (clear) o bit 3 de `x`, sem alterar os demais.
- d) Alternar (toggle) os bits 0 e 1 de `x`, sem alterar os demais.
- e) Verificar se o bit 7 de `x` está ligado (a expressão deve resultar em verdadeiro ou falso).

Convenção: bits numerados da direita para a esquerda, começando em 0 (bit 0 = LSB, bit 7 = MSB).

Resposta — Questão 3

- a) `x & 0xF0`
- b) `x | (1 << 5)` ou `x | 0x20`
- c) `x & ~ (1 << 3)` ou `x & 0xF7`
- d) `x ^ 0x03`
- e) `(x & 0x80) != 0` ou `x & (1 << 7)`

Questão 4 — Leitura de código

Analise cada trecho de código e determine o valor final de `resultado` em binário e hexadecimal.

- a)

```
unsigned char flags = 0b10010100;
unsigned char resultado = flags & 0x0F;
```
- b)

```
unsigned char reg = 0b01100001;
unsigned char resultado = reg | (1 << 7);
```
- c)

```
unsigned char estado = 0b11111111;
unsigned char resultado = estado & ~ (1 << 4);
```
- d)

```
unsigned char valor = 0b10101010;
unsigned char resultado = valor ^ 0b11001100;
```

e)

```
unsigned char a = 0xA3;  
unsigned char b = 0x5C;  
unsigned char resultado = (a & 0xF0) | (b & 0x0F);
```

Resposta — Questão 4

- a) $0b00000100 = 0x04$
- b) $0b11100001 = 0xE1$
- c) $0b11101111 = 0xEF$
- d) $0b01100110 = 0x66$
- e) $0b10101100 = 0xAC$

Questão 5 — Escrita de funções

Implemente as funções abaixo usando **apenas** operadores bitwise (&, |, ^, ~, «, »).

- a) Uma função que recebe um byte e retorna 1 se o bit na posição `pos` estiver ligado, e 0 caso contrário.

```
int bit_ligado(unsigned char byte, int pos) {  
    // sua implementacao  
}
```

- b) Uma função que recebe um byte e uma posição, e retorna o byte com o bit naquela posição **ligado**.

```
unsigned char ligar_bit(unsigned char byte, int pos) {  
    // sua implementacao  
}
```

- c) Uma função que recebe um byte e uma posição, e retorna o byte com o bit naquela posição **desligado**.

```
unsigned char desligar_bit(unsigned char byte, int pos) {  
    // sua implementacao  
}
```

- d) Uma função que recebe um byte e troca os 4 bits mais significativos com os 4 bits menos significativos (swap nibbles).

```
unsigned char swap_nibbles(unsigned char byte) {  
    // sua implementacao  
}
```

Resposta — Questão 5

a)

```
int bit_ligado(unsigned char byte, int pos) {  
    return (byte >> pos) & 1;  
}
```

b)

```
unsigned char ligar_bit(unsigned char byte, int pos) {  
    return byte | (1 << pos);  
}
```

c)

```
unsigned char desligar_bit(unsigned char byte, int pos) {  
    return byte & ~(1 << pos);  
}
```

d)

```
unsigned char swap_nibbles(unsigned char byte) {  
    return (byte << 4) | (byte >> 4);  
}
```